PROSPECT: PROACTIVE SPECULATION FOR LOSSLESS ACCELERATION OF MULTI-AGENT SYSTEMS

Anonymous authors
Paper under double-blind review

ABSTRACT

Multi-agent LLM systems are increasingly used for complex, long-horizon tasks, but their sequential, dependent calls create high end-to-end latency. Speculative execution reduces latency *losslessly*, by pre-executing likely future calls and committing only those the system would have made anyway. Extending it from a single agent to a multi-agent system is hard because a speculation must predict which agents are invoked next in an execution trace that is a DAG instead of a chain, and what each predicted agent receives. An agent starts early only if every incoming edge is predicted correctly. We present ProSpect, which separates these two predictions. Choosing the next agent is cast as a link prediction task over a graph of agents assembled from past traces and answered by a recurrence folded along the execution DAG; the Speculator LLM is left with the task of message generation alone for the predicted agent, and boosted further through retrieval-augmented in-context learning on Actor's past traces. The speculation budget between these two prediction heads is split between them adaptively, by the predictor's confidence. Across four multi-agent systems and spanning the Qwen, Gemma and DeepSeek LLM families, ProSpect saves up to $\approx 38\%$ of wall-clock time and outperforms the state of the art speculative acceleration method on every system, while also reducing token cost and compute flops.

1 INTRODUCTION

Large language model (LLM) based multi-agents now carry out long-horizon tasks inside interactive environments such as operating systems, browsers, code repositories, and research pipelines (Ye et al., 2026). Each step of such an agent is a blocking call: an LLM invocation, a tool or MCP request, a human query, or a message to another agent. The next step starts only after the current one returns, making wall-clock time a bottleneck, with reported task times running from minutes to hours across computer use (Abhyankar et al., 2026), deep research (OpenAI, 2025), software development (Hong et al., 2024; Qian et al., 2024), and ML research (Schmidgall et al., 2025). The problem worsens as agents adopt larger models with stronger reasoning, since each call then takes longer: MetaGPT takes 75 s with GPT-3.5 but 553 s with GPT-4 (Hong et al., 2024), and Agent Laboratory 19 min with GPT-4o but 103 min with o1-preview (Schmidgall et al., 2025). For a growing set of deployments, lost wall-clock time is consequential well beyond user annoyance. In security operations, an intrusion spreads through the network while the system deliberates. In trading, a price gaps away before the decision lands, so a correct-but-late answer is worth less than a timely one, and sometimes worth nothing. This raises the central question of this paper: *can wall-clock time be cut without asking a system to trade away the very quality it was built to protect?*

1.1 RELATED WORK

The latency of an agentic system is dominated by its LLM calls, and existing remedies fall into two groups, each giving up one of the two things that matter. The first trades output for speed. *LLM routing and cascades* send a query to a cheaper model, or escalate only when its answer looks unreliable (Chen et al., 2024; Ding et al., 2024; Ong et al., 2025); *communication pruning* drops redundant messages between agents (Zhang et al., 2025a); *speculative reasoning* drafts reasoning steps and verifies them approximately (Wang et al., 2026; Fu et al., 2025). All are faster, but a weaker model, thinner context, or an approximate check can change what the system returns. The second preserves the output but cannot cross a dependency. *Speculative decoding* has a small model draft tokens that the target LLM verifies in parallel, leaving the output distribution intact (Leviathan et al., 2023); *serving optimizations* expose inter-request dependencies to the scheduler (Lin et al., 2024);

Reviewers: please read the Reviewer Guidelines (iclr.cc/Conferences/2027/ReviewerGuidelines) and the AI Policy for Reviewers (iclr.cc/Conferences/2027/AIPolicyForReviewers). If you used AI to expand, edit, or polish your review, please provide the input text to the LLM. Better still, consider skipping the LLM and submitting your original text: we, and the authors, are much more interested in your unedited thoughts than in what an LLM has to say. AI-assisted or not, you are putting your name and reputation behind your review: LLM-generated falsehoods, hallucinations or misrepresentations are subject to disciplinary action, which may include desk-rejecting all papers you have authored.

parallel function calling executes independent calls concurrently (Kim et al., 2024). Each shortens a call, or overlaps calls that are already free to run, but a call that waits on an earlier output still waits.

Lossless speculative acceleration. *Can we accelerate a pipeline while **guaranteeing** the output remains unaffected?* Borrowing from speculative execution in processors (Tomasulo, 1967; Estebanez et al.), *speculative actions* (Ye et al., 2026) answer yes by separating two roles. The *Actor* is the authoritative model the pipeline normally runs, a large LLM that is accurate but slow, and its outputs define the result. The *Speculator* is a much smaller model that receives the same prompt, finishes long before the Actor does, and guesses the Actor's output and hence the call that will follow. It is asked for k guesses, out of which only one has to be right, and all k follow-up calls are issued at once on the Actor's own model. While the Actor is still finishing the *current* call, the k speculative copies of the *next* call already run beside it. When the current call returns, the follow-up it genuinely requires is compared against those k. If one matches exactly, that follow-up is already part-way done and the pipeline adopts it. If none matches, the genuine follow-up is issued as usual. Since every committed call is one the Actor would have made, the method is *lossless*. Note that PROSPECT is breadth-focused speculation: at every round, the Speculator makes k guesses for the Actor's next output. There is another line of work on depth-wise speculation, where a fast model drafts several future steps along one branch for the Actor to verify (Hua et al., 2025; Guan et al., 2026). Depth looks several steps ahead but saves nothing after the first miss, whereas breadth spends its k candidates on a single step, therefore two approaches are complementary rather than competing.

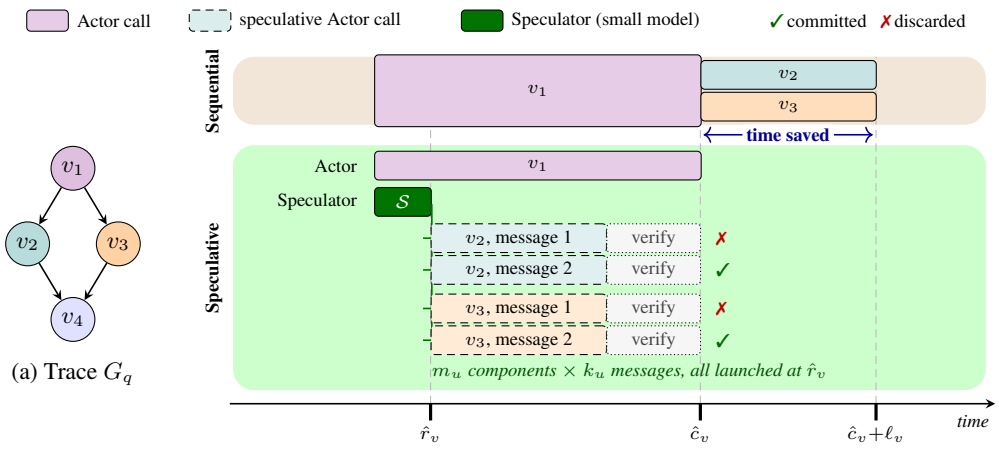
Limitations of Speculative Acceleration for MAS: When speculative actions (Ye et al., 2026) are applied to multi-agent systems (MAS), four difficulties emerge that have no single-agent counterpart. We illustrate them on the execution of Figure 1(a), where a planner v_1 invokes two workers v_2 and v_3 whose outputs both feed a reviewer v_4.

- **Compound prediction.** A speculation succeeds only if the Speculator names the right next agent *and* its exact prompt, so the two accuracies multiply.
- **Parallel branches.** A step may invoke several agents whose outputs are later merged. The reviewer v_4 can start early only if the speculations for *both* workers are correct; a correct guess for v_3 alone brings no benefit, since v_4 still waits on v_2. The Speculator must therefore predict the whole set of invoked agents, and the chance of a useful speculation falls as more branches must be right at once.
- **Speculation latency.** A speculation helps only if it finishes well before the current call returns. Since the prompt depends on which agent receives it, the Speculator must settle the agent first and then write a prompt for each candidate, work that grows with the number of candidates and can consume much of the time it is trying to save.
- **Token cost.** Choosing the next agent is a selection within a fixed and recurring set, yet an LLM Speculator solves it by generation, reading every agent description and the interaction history at each step. This costs tokens and compute that grow with the number of agents, and it is spent even on speculations that are later discarded.

1.2 CONTRIBUTIONS

To address these challenges, we design PROSPECT. Our core contributions are as follows.

- **Problem formulation.** We formalize lossless speculative acceleration for MAS, where an execution is a DAG rather than a chain. Two consequences follow that have no single-agent counterpart: the speculation budget must be split between predicting *which* components are invoked and *what* each of them receives, and a node can start early only if every one of its incoming edges was predicted correctly. We give a latency model that exposes both effects and confirms that verification-based commitment stays lossless on a DAG.
- **Methodology.** PROSPECT decouples the two predictions a speculation requires. Choosing the next components is cast as *link prediction* over a graph of components assembled from the traces a deployed MAS already produces, and answered by a lightweight graph network in one forward pass; so routing costs no LLM call and concurrent branches are predicted together. The Speculator LLM is left with the generative half alone, and empowered with in-context learning through retrieval augmented generation.
- **Empirical validation.** Across four MAS and six Actor-Speculator pairings spanning the Qwen, Gemma and DeepSeek families, PROSPECT saves 17.25% to 37.86% of wall-clock time over sequential execution, improving on the state-of-the-art speculative actions (Ye et al., 2026) by 6.0% to 28.1% in every paired comparison, while also lowering token and compute costs.



(a) Trace G_q

(b) Timeline of the step that follows v_1

Figure 1: Speculative execution of the running example. **(a)** The planner v_1 invokes the workers v_2 and v_3, whose outputs both feed the reviewer v_4. **(b)** Sequentially, v_2 and v_3 start only once v_1 returns. Under speculation the Speculator returns by $\hat{r}_v$ while v_1 is still running, and its $m_u \times k_u$ candidates are launched at once as ordinary Actor calls, so they have finished and are waiting when v_1 returns at $\hat{c}_v$. Because v_4 merges both branches, it starts early only if v_2 *and* v_3 are both hits.

2 Lossless Speculative Execution in Multi-Agent Systems

In this section, we rigorously set up the framework of speculative acceleration for MAS, the lossless latency optimization problem, and theoretical guarantees that characterize output fidelity and latency. All notations used in our work are summarized in Table 4 of the Appendix.

Definition 1 (Multi-agent system)**.** *A* multi-agent system *(MAS) is a tuple $\mathcal{M} = (\mathcal{A}, \mathcal{T}, C)$, where $\mathcal{A}$ is a set of agents and $\mathcal{T}$ a set of tools. We refer to their union $\mathcal{U} = \mathcal{A} \cup \mathcal{T} = \{1, \dots, n\}$ as* components*. Component i maps an input x to an output $y \sim F_i(\cdot \mid x)$, sampled from an LLM if i is an agent and returned by an external function or API if i is a tool. The* coordination mechanism *C maps the outputs produced so far to the next* invocations *$z = (i, x)$, each specifying the receiving component $i \in \mathcal{U}$ and its input x.*

Components and C are black boxes. C captures the native control flow of the system, such as workflow code, agent-generated handoffs, and manager-based speaker selection; routing belongs to C even when it is itself an LLM call, so the nodes of an execution are the invocations that carry out the task.

Definition 2 (Execution trace)**.** *The execution of $\mathcal{M}$ on a task q is a directed acyclic graph $G_q = (V_q, E_q)$. Each node $v \in V_q$ is an invocation $z_v = (i_v, x_v)$ with output y_v and latency $\ell_v > 0$. An edge $(u, v) \in E_q$ carries a* message *$y_{u \to v}$, the part of the output y_u that C routes to v. Writing $\mathcal{P}(v)$ for the parents of v, the input of v is $x_v = \{q, \sigma\} \cup \{y_{u \to v} \mid u \in \mathcal{P}(v)\}$ where σ stands for context fixed before execution.*

Iterative interactions, such as debate rounds or revision loops, are unrolled over time, so G_q is acyclic. A node with several children sends a message to each of them, and a node with several parents assembles the messages it receives. As a running example we use the trace of Fig. 1(a): a planner v_1 invokes two workers v_2 and v_3, whose outputs both feed a reviewer v_4.

Definition 3 (Non-speculative latency)**.** *In standard execution, v can start only after all its parents finish, since no incoming message exists until then. With e_u denoting the end time of u,*

$$r_v = \max_{u \in \mathcal{P}(v)} e_u, \qquad e_v = r_v + \ell_v, \qquad L(G_q) = \max_{v \in V_q} e_v, \tag{1}$$

where r_v is the ready time *of v ($r_v = 0$ for source nodes), at which v starts, ℓ_v is its latency, and $L(G_q)$ is the end-to-end latency.*

2.1 Speculative Execution

How speculation buys time. Fig. 1(b) instantiates speculation on the trace of Fig. 1(a). While the planner v_1 is still running, the Speculator predicts which components it will invoke and what each of them receives, and every candidate is launched at once at the Actor's own fidelity, so that a correct guess is already computed when v_1 returns and is committed once the check confirms it.

One round of speculation ends there. The workers v_2 and v_3 are now confirmed, and the Speculator starts afresh on each of them, in exactly the same way, to guess what *they* will invoke. Speculation is therefore *one-hop bounded*: at any instant it looks exactly one invocation beyond what the Actor has confirmed, and never speculates from a speculation.

DEFINITION 4 (Speculator and speculation width)**.** *At the moment the invocation of a node u is confirmed and its input becomes known, the Speculator $\mathcal{S}$ predicts the components that u will invoke next together with the message each of them receives,*

$$\widehat{E}_u = \left\{ \left(\hat{i}^{(a)},\, \hat{y}^{(a,b)}\right) : a \in [m_u],\; b \in [k_u] \right\}, \tag{2}$$

where $\hat{i}^{(a)}$ is a candidate child component and $\hat{y}^{(a,b)}$ a candidate message from u to $\hat{i}^{(a)}$. The candidates become available within δ_u of that moment, where δ_u is the latency of the Speculator.

The two indices are the two things the Speculator shadows. Its width on C is m_u, the number of candidate next components, and its width on the Actor is k_u, the number of candidate messages it writes for each of them. Hence, the *speculation width* of u is $w_u = |\widehat{E}_u| = m_u \times k_u$.

DEFINITION 5 (Speculation hit)**.** *An edge $(u, v) \in E_q$ is a* hit *if it was predicted correctly when speculating on u, that is, if some candidate $(\hat{i}, \hat{y}) \in \widehat{E}_u$ satisfies $\hat{i} = i_v$ and $\hat{y} = y_{u\to v}$. A node v is a* hit *if all of its incoming edges are hits.*

A node is therefore a hit only if every branch feeding it is predicted correctly, both the component invoked and the message it receives. In the running example, a correct guess on the edge (v_3, v_4) alone does not let the reviewer v_4 start early: it still waits for the worker v_2.

Commit rule. At time r_v, the realized invocation z_v becomes known. If speculative calls were launched on exactly $z_v = (i_v, x_v)$, the earliest of them is committed as the execution of v and its output is taken as y_v; otherwise the Actor executes z_v. All remaining speculative calls are discarded.

Three moments per node. In the non-speculative pipeline, the ready time r_v of a node is both the moment its invocation is confirmed and the moment it starts. Speculation pulls these apart into three moments, drawn in Fig. 1(b): the *confirmation time* $\hat{c}_v$, at which every parent has returned and z_v becomes known; the *start time* $\hat{r}_v$, at which v is launched on an input that could still be predicted, which on a hit is δ_u after u was confirmed and on a miss slides right to $\hat{c}_v$; and the *end time* $\hat{e}_v$, at which its output may be used. A hit is therefore what separates $\hat{r}_v$ from $\hat{c}_v$; without one the two coincide and v behaves as in non-speculative execution (Eq. 1).

DEFINITION 6 (Speculative latency)**.** *Let $t_{u\to v}$ be the time at which the correct message on edge (u, v) becomes available,*

$$t_{u\to v} = \begin{cases} \min\left(\hat{e}_u,\; \hat{c}_u + \delta_u\right) & \text{if } (u,v) \text{ is a hit,} \\ \hat{e}_u & \text{otherwise.} \end{cases} \tag{3}$$

If the Speculator's prediction is a miss, we discard the speculative progress and initiate the next agent's call. If it predicts the edge correctly, the message is available once the Speculator, invoked at $\hat{c}_u$, returns δ_u later, unless the Actor is faster still, in which case it reduces to $\hat{e}_u$. A speculative call on v starts once all of its incoming messages are available. Hence,

$$\hat{c}_v = \max_{u\in\mathcal{P}(v)} \hat{e}_u, \qquad \hat{r}_v = \max_{u\in\mathcal{P}(v)} t_{u\to v}, \qquad \hat{e}_v = \max\left(\hat{r}_v + \ell_v,\; \hat{c}_v\right), \qquad \hat{L}(G_q;\mathcal{S}) = \max_{v\in V_q} \hat{e}_v. \tag{4}$$

The two terms of $\hat{e}_v$ are the two prerequisites for the output of v to be usable: the call must have finished, and the prediction it was launched on must have been verified, which happens only once every parent has returned. Since $t_{u\to v} \le \hat{e}_u$ we have $\hat{r}_v \le \hat{c}_v$, with equality when no incoming edge of v is a hit, in which case nothing was launched early and $\hat{e}_v = \hat{c}_v + \ell_v$. If no edge of G_q is a hit, then $\hat{c}_v = \hat{r}_v = r_v$ at every node and Eq. 4 reduces to Eq. 1.

PROBLEM 1 (Lossless speculative acceleration of MAS)**.** *Given a MAS $\mathcal{M}$, a task distribution $\mathcal{D}$, and a speculation width cap W, find a Speculator $\mathcal{S}$ that minimizes the expected end-to-end latency*

$$\min_{\mathcal{S}}\; \mathbb{E}_{q\sim\mathcal{D}}\left[\hat{L}(G_q;\mathcal{S})\right] \quad \textit{s.t.} \quad w_u = m_u k_u \le W \quad \forall u \in V_q, \tag{5}$$

where outputs are committed under the commit rule of §2.1. The expectation is also taken over the sampling of component outputs.

2.2 Theoretical Characterization

Ye et al. (2026) introduce the Actor and Speculator roles, and show that verification-based commitment preserves the output distribution, and bound the achievable latency reduction at 50% for a single agent issuing one call at a time. Both results carry over to MAS where the execution trace is a DAG instead of a linear chain.

Proposition 1 (Losslessness)**.** *Speculative pipeline does not change the output of the non-speculative pipeline. More specifically, under the commit rule, the joint distribution of the execution trace G_q and its outputs $(y_v)_{v\in V_q}$ is the same under speculative and non-speculative execution.*

Proof. By the commit rule, every node v of the realized trace is executed exactly once, by a call to component i_v on input x_v: either the Actor's own call, or a speculative call launched on exactly $z_v = (i_v, x_v)$, retained only once z_v is known. Speculation therefore alters only *when* that call is issued. By Def. 1, the output of a call on $z_v = (i_v, x_v)$ is a draw from $F_{i_v}(\cdot \mid x_v)$. Hence, conditioned on z_v, the committed y_v is a draw from $F_{i_v}(\cdot \mid x_v)$ in both executions. Since C constructs every invocation from committed outputs alone, induction along a topological order of G_q gives the claim. □

Proposition 2 (Latency bounds)**.** *Speculation never increases latency, and can reduce it by at most 50%. Specifically, let d be the* depth *of the MAS trace G_q, the number of nodes on its longest path, and suppose every node has latency ℓ. Then for every Speculator $\mathcal{S}$, every pattern of hits and misses, and every $\delta_u \geq 0$,*

$$\left\lceil \tfrac{d}{2} \right\rceil \ell \;\leq\; \hat{L}(G_q; \mathcal{S}) \;\leq\; L(G_q) = d\,\ell. \tag{6}$$

Proof. See Appendix A.

3 ProSpect: Proposed Methodology

Every speculation at a node u answers two questions at once, and Def. 5 gives nothing for getting only one of them right. The two are not of the same kind: which components u invokes next is a choice within $\mathcal{U}$, a small set whose transitions repeat across runs, whereas what each of them receives is generation. ProSpect therefore answers them with two modules, split along the two black boxes of Def. 1. The *component predictor* $\mathcal{S}_C$ shadows the coordination mechanism, predicting which components u will invoke as *link prediction* over a graph of components assembled from past traces, answered by a forward pass rather than a generation. The *message writer* $\mathcal{S}_F$ shadows the output distribution F_{i_u}: told which component to write for, it produces the message u would send to it.

We next discuss how ProSpect builds $\mathcal{S}_C$, $\mathcal{S}_F$ and decides on the speculative width m_u and k_u.

3.1 Component Prediction

Training Set. ProSpect assumes access to a corpus $\mathcal{H}$ of past executions of $\mathcal{M}$, produced by the non-speculative pipeline. $\mathcal{H}$ supplies the supervision for $\mathcal{S}_C$.

The prediction task. At a node u the Speculator must name the set of components that u invokes. Let $\mathcal{C}(u)$ denote the children of u in G_q, with $\mathcal{C}(u) = \{\perp\}$ when u invokes nothing further. Because $\mathcal{U}$ is fixed and the same system is run again and again, predicting $\mathcal{C}(u)$ before u returns is a **link prediction** problem over components: the vertices are known in advance, and the question is which edges leave u next.

Scores and objective. Let $\mathcal{N}(i)$ denote the set of components that i invoked at least once in the trainset $\mathcal{H}$. ProSpect scores the candidate set $\mathcal{A}(u) = \mathcal{N}(i_u) \cup \{\perp\}$. Towards that objective, let $\mathbf{h}_j \in \mathbb{R}^{d_h}$ be a learned embedding of component j, let $\mathbf{w}_{ij}$ summarize how habitual the transition from i to j has been, and let $\mathbf{g}_u$ collect what is known about u at the moment it becomes ready. A candidate is scored by concatenating the three and reading off a scalar,

$$p_{uj} = \texttt{Sigmoid}\Big(\mathbf{W}_2\,\texttt{ReLU}\big(\mathbf{W}_1\big[\,\mathbf{g}_u \,\|\, \mathbf{h}_j \,\|\, \mathbf{w}_{i_u j}\,\big]\big)\Big), \qquad j \in \mathcal{A}(u). \tag{7}$$

Training fits these scores to the routing decisions the traces already record: the realized children of u are the positives and the remaining candidates the negatives, so ProSpect minimizes:

$$\mathcal{L} = -\sum_{G_q\in\mathcal{H}} \sum_{u\in V_q} \left(\sum_{j\in\mathcal{C}(u)} \log p_{uj} \;+ \sum_{j\in\mathcal{A}(u)\setminus\mathcal{C}(u)} \log\big(1 - p_{uj}\big) \right). \tag{8}$$

We next discuss how the embeddings, namely $\mathbf{w}_{ij}$, $\mathbf{g}_u$ and $\mathbf{h}_j$ are formulated.

Component embeddings. Component embeddings are free parameters rather than functions of any input feature: the model holds a lookup table $\mathbf{H} \in \mathbb{R}^{(|\mathcal{U}|+1)\times d_h}$ with one row per component and one for $\perp$, initialized randomly and trained end to end by Eq. 8.

Priors from past executions ($\mathbf{w}_{ij}$). The sets $\mathcal{N}(i)$ and their statistics form a static *interaction graph* $\mathcal{G} = (\mathcal{U} \cup \{\perp\}, \mathcal{E})$, with $(i, j) \in \mathcal{E}$ whenever $j \in \mathcal{N}(i)$ and every component carrying an edge to $\perp$. The weight of an edge, $w_{ij} = \frac{n_{ij}}{n_i}$, is the fraction of invocations of i whose children included j; n_{ij} denotes the number of invocations of i that invoked j and n_i is the number of invocations of i.

A rate alone is not the whole of what $\mathcal{G}$ knows about a transition, since $w_{ij} = 0.5$ from two observations and from two hundred are the same rate but not the same evidence. An MLP_e, learned jointly with the rest of the model, reads the rate together with its support, to model both factors jointly.

$$\mathbf{w}_{ij} = \mathrm{MLP}_e\big(\big[\, w_{ij},\ \log(1 + n_{ij}) \,\big]\big). \tag{9}$$

Context from the current execution ($\mathbf{g}_u$). What $\mathbf{g}_u$ must carry is everything this execution has revealed by the time u becomes ready, namely the prefix of G_q that reaches u: a set of traversed edges together with the partial order in which they were traversed. The building block is a single edge. Let ϕ be a frozen text encoder, applied to the task prompt q and to internal messages among components. At the ready time of u, each parent $p \in \mathcal{P}(u)$ contributes a fully observed edge embedding:

$$\mathbf{e}_{p\to u} = \big[\, \mathbf{h}_{i_p} \,\|\, \phi(y_{p\to u}) \,\|\, \mathbf{w}_{i_p i_u} \,\big], \tag{10}$$

which encodes the parent's embedding, the message it sent, and how habitual that transition is.

While $\mathbf{h}_{i_u}$ is a fixed embedding of an agent, we now generate a context-dependent embedding for u, by performing an attention-weighted aggregation over all incoming edges. Formally,

$$\alpha_{pu} = \frac{\exp\big(\langle \mathbf{W}_Q \mathbf{h}_{i_u},\ \mathbf{W}_K \mathbf{e}_{p\to u}\rangle / \sqrt{d_k}\big)}{\sum_{p'\in\mathcal{P}(u)} \exp\big(\langle \mathbf{W}_Q \mathbf{h}_{i_u},\ \mathbf{W}_K \mathbf{e}_{p'\to u}\rangle / \sqrt{d_k}\big)}, \qquad \mathbf{m}_u = \sum_{p\in\mathcal{P}(u)} \alpha_{pu}\, \mathbf{e}_{p\to u}. \tag{11}$$

Here $\mathbf{W}_Q$ and $\mathbf{W}_K$ are learned query and key projections into a common space of dimension d_k.

$\mathbf{m}_u$ captures the immediate context available at u, but not the entire trace prior to it. A recurrent encoder is the natural way to embed this trace. Recurrence as it is ordinarily written, however, assumes a *linear* history. For example, $\mathbf{s}_t = \mathrm{GRU}(\mathbf{s}_{t-1}, \mathbf{x}_t)$ presumes a unique predecessor at every step. In an MAS execution trace, a node may be reached through several branches that ran concurrently, and G_q fixes only a partial order among its edges. Hence, there is no unique $\mathbf{s}_{t-1}$. ProSpect therefore folds along the DAG itself. Specifically,

$$\mathbf{s}_u = \mathrm{GRU}\Big(\textstyle\sum_{p\in\mathcal{P}(u)} \alpha_{pu}\, \mathbf{s}_p,\ \ \mathbf{m}_u\Big), \tag{12}$$

with $\mathbf{s}_u = \mathbf{s}_0$ a learned vector and $\mathbf{m}_u = \mathbf{0}$ at a source node, which has no parents. Reusing α_{pu} across both pools says that how much a parent matters to u is a property of the parent rather than of which of its attributes is read. The context of Eq. 7 is then $\mathbf{g}_u = \big[\, \phi(q) \,\|\, \mathbf{h}_{i_u} \,\|\, \mathbf{s}_u \,\big]$.

3.2 Speculation with Adaptive Width

With $\mathcal{S}_C$ trained, the obvious rule is to predict every candidate whose score exceeds 0.5 and give each of the m_u survivors W/m_u message speculations through $\mathcal{S}_F$. The rule respects the budget in neither direction: it can pass more candidates than the cap allows, leaving W/m_u below one prompt, or none at all, leaving the width unused.

ProSpect therefore reads the ranking rather than the absolute scores. Independent sigmoids do not form a distribution, so we normalize them over the candidate set, $\bar{p}_{uj} = p_{uj} / \sum_{j'\in\mathcal{A}(u)} p_{uj'}$, and write $\bar{p}_{(1)} \geq \bar{p}_{(2)} \geq \cdots$ for the result in decreasing order. Walking down this order, ProSpect takes the fewest candidates whose cumulative mass reaches a target τ, and stops at W if it does not. We then set $k_u = floor(W/m_u)$.

3.3 Message Speculation

Let $\hat{i}$ denote one of the predicted components at node u. The message writer $\mathcal{S}_F$ is asked for the message agent i_u would send to agent $\hat{i}$, given the task prompt q, the input message x_u to i_u, and

Table 1: MASs, task sets, and trace statistics. $|\mathcal{U}|$ is the number of components, and d the mean depth of an execution trace. GAIA is detailed in Mialon et al. (2024).

System	Domain	Coordination	$\lvert\mathcal{U}\rvert$	Task set (#tasks)	d
Co-Sight (Zhang et al., 2025b)	Web and files	Parallel experts, verifier	23	GAIA (165)	20.16
Magentic-One (Fourney et al., 2024)	Web and files	Orchestrator, four workers	17	GAIA (165)	23.20
AILA (Mandal et al., 2025)	Microscopy	Planner, handlers, tools	7	AFMBench (100)	3.10
TeLLAgent (Sun et al., 2026)	Molecular design	Supervisor, executor, tools	16	Tool-calling suite (100)	7.88

the role of i_u. Extending Ye et al. (2026), drawing k_u samples per predicted component $\hat{i}$ yields the $m_u k_u = W$ pairs of Def. 4.

Retrieved demonstrations for in-context learning. To help $\mathcal{S}_F$ shadow the Actor more closely, PROSPECT shows it how i_u has answered similar questions before. Let $\mathcal{H}(i, j)$ denote the set of edges in the training corpus on which agent i invoked agent j; each such edge carries the input that invocation received and the message it sent onward. Since similar inputs tend to elicit similar outputs, a past call *from* i_u *to* $\hat{i}$ whose input resembles x_u is evidence about the message the Actor may produce on that very edge. PROSPECT therefore ranks the edges of $\mathcal{H}(i_u, \hat{i})$ by the cosine similarity of their input to x_u under the encoder ϕ, and places the input-message pairs of the top-κ into the prompt alongside x_u, where κ is the context budget.

4 EXPERIMENTS

Our evaluation is organized around two questions.

- **RQ1.** How much latency does PROSPECT save against sequential execution and against speculative actions (Ye et al., 2026) extended to a MAS by asking the Speculator LLM to name the next agent first and then write its message?
- **RQ2.** Why does PROSPECT work? We ablate the predictor, separate component accuracy from message accuracy, and various other parameters.

Our implementation is available at `https://anonymous.4open.science/r/ProSpect`.

4.1 EXPERIMENTAL SETUP

Systems and tasks. We evaluate on four MAS chosen for different coordination structures: parallel experts, centralized orchestration, nested agent-to-tool calls, and supervisor-executor loops with recovery. Table 1 summarizes each system, the task set we run it on, and the trace statistics that bear on speculation. We use each benchmark in full, with no task-level filtering. We leave each system's orchestration logic unchanged and use one recording and replay procedure throughout. For every task we run the Actor once and record the components it invokes, their messages and tool calls, token usage, and per-step wall-clock latency. Every method is then evaluated against this same recording, with the Speculator executed live.

Baseline. There is only one work on breadth-focused speculative acceleration for agentic systems Ye et al. (2026). We call this method *SA (Speculative Action)*: the Speculator returns W joint candidates, each naming a component and its message.

Results are reported under 10-fold cross-validation over the task set of each system. App. B gives further details on default parameters, per-system stopping rules, the sandboxing setup, and the software-hardware environment.

4.2 LATENCY, HIT RATE AND TOKEN CONSUMPTION

Tables 2 and 3 present results across the Qwen, Gemma and DeepSeek families.

Latency. PROSPECT reduces end-to-end latency on every system and every backend, and outperforms speculative actions (SA) in every paired comparison. PROSPECT's advantage over SA is largest on the two scientific systems, TeLLAgent and AILA, and this is driven by SA's weak next-agent prediction there (4.89 to 32.67%, against 33.68 to 92.96% on the two web systems). Our hypothesis is that naming the next component by generation relies on the Speculator's prior over component roles, which is better served by the generic roles of a web-browsing system than by instrument handlers or molecular property tools.

Hit Rate. The significantly superior latency saving of PROSPECT can be attributed to its superior hit rates. Specifically, in next-agent prediction, PROSPECT is dramatically superior in all MASs except Magentic-One, where they obtain similar numbers. Furthermore, owing to retrieval augmented

Table 2: Latency, hit rate, and cost for the three Actor–Speculator pairings named in block headers (a)–(c). Cost denotes the increase in Actor tokens over non-speculative execution, the Speculator's tokens per question, and the increase in FLOPs over sequential Actor execution. Shading highlights the better arm per system.

System	Arm	Time	Next-agent	Content	Hit	Cost		
		saved (%)↑	match (%)↑	match (%)↑	rate (%)↑	Actor (+%)↓	Speculator↓	FLOPs (+%)↓
(a) Actor: Qwen 3.6 27B, thinking on | Speculator: Qwen 3.5 9B, thinking off								
Co-Sight	SA	$13.58_{\pm 12}$	$41.39_{\pm 21}$	$33.05_{\pm 22}$	$24.93_{\pm 20}$	$99.40_{\pm 36}$	$7373_{\pm 10569}$	$242.08_{\pm 51}$
	ProSpect	$23.80_{\pm 13}$	$78.48_{\pm 15}$	$47.57_{\pm 24}$	$41.07_{\pm 23}$	$94.46_{\pm 38}$	$8473_{\pm 13842}$	$214.49_{\pm 51}$
TeLLAgent	SA	$1.10_{\pm 3}$	$18.55_{\pm 23}$	$27.97_{\pm 21}$	$4.07_{\pm 8}$	$147.86_{\pm 48}$	$3738_{\pm 1741}$	$282.95_{\pm 38}$
	ProSpect	$21.62_{\pm 14}$	$90.23_{\pm 18}$	$59.48_{\pm 30}$	$56.65_{\pm 30}$	$105.83_{\pm 47}$	$2245_{\pm 1678}$	$201.21_{\pm 58}$
Magentic-One	SA	$21.45_{\pm 13}$	$95.40_{\pm 7}$	$46.39_{\pm 22}$	$43.47_{\pm 23}$	$101.07_{\pm 38}$	$6474_{\pm 6410}$	$206.20_{\pm 52}$
	ProSpect	$27.47_{\pm 13}$	$87.16_{\pm 12}$	$62.68_{\pm 20}$	$57.63_{\pm 19}$	$87.89_{\pm 22}$	$13034_{\pm 13616}$	$193.05_{\pm 44}$
AILA	SA	$9.80_{\pm 15}$	$12.73_{\pm 17}$	$12.39_{\pm 16}$	$12.39_{\pm 16}$	$77.24_{\pm 36}$	$251.59_{\pm 150}$	$86.27_{\pm 38}$
	ProSpect	$37.86_{\pm 20}$	$60.15_{\pm 13}$	$41.37_{\pm 14}$	$41.37_{\pm 14}$	$63.85_{\pm 36}$	$406.82_{\pm 599}$	$74.81_{\pm 41}$
(b) Actor: Qwen 3.6 27B, thinking on | Speculator: Qwen 3.5 2B, thinking off								
Co-Sight	SA	$9.18_{\pm 10}$	$33.68_{\pm 17}$	$20.28_{\pm 15}$	$12.08_{\pm 11}$	$123.79_{\pm 38}$	$5599_{\pm 6028}$	$246.48_{\pm 41}$
	ProSpect	$22.48_{\pm 14}$	$82.64_{\pm 14}$	$37.51_{\pm 22}$	$34.65_{\pm 22}$	$108.33_{\pm 41}$	$6465_{\pm 8191}$	$212.38_{\pm 55}$
TeLLAgent	SA	$1.09_{\pm 4}$	$4.89_{\pm 12}$	$17.55_{\pm 24}$	$3.30_{\pm 12}$	$174.49_{\pm 51}$	$3903_{\pm 2298}$	$260.06_{\pm 35}$
	ProSpect	$19.93_{\pm 14}$	$89.99_{\pm 18}$	$49.59_{\pm 31}$	$48.49_{\pm 30}$	$113.58_{\pm 50}$	$3580_{\pm 2202}$	$181.93_{\pm 58}$
Magentic-One	SA	$7.36_{\pm 8}$	$84.93_{\pm 9}$	$16.88_{\pm 12}$	$14.18_{\pm 10}$	$136.52_{\pm 39}$	$6790_{\pm 5956}$	$245.20_{\pm 46}$
	ProSpect	$17.28_{\pm 12}$	$80.03_{\pm 11}$	$37.41_{\pm 17}$	$27.57_{\pm 10}$	$112.62_{\pm 35}$	$13693_{\pm 12103}$	$198.71_{\pm 50}$
AILA	SA	$12.64_{\pm 17}$	$17.56_{\pm 20}$	$16.48_{\pm 20}$	$16.48_{\pm 20}$	$76.06_{\pm 38}$	$130.09_{\pm 98}$	$76.98_{\pm 38}$
	ProSpect	$37.52_{\pm 19}$	$60.55_{\pm 13}$	$39.23_{\pm 13}$	$39.23_{\pm 13}$	$65.00_{\pm 37}$	$352.96_{\pm 494}$	$67.27_{\pm 37}$
(c) Actor: Gemma 31B | Speculator: Gemma E2B, thinking off								
Co-Sight	SA	$13.15_{\pm 11}$	$47.03_{\pm 22}$	$30.42_{\pm 20}$	$22.83_{\pm 18}$	$146.25_{\pm 46}$	$5872_{\pm 6453}$	$229.67_{\pm 47}$
	ProSpect	$23.24_{\pm 12}$	$81.35_{\pm 13}$	$47.13_{\pm 21}$	$40.69_{\pm 20}$	$125.94_{\pm 42}$	$5310_{\pm 5685}$	$187.28_{\pm 46}$
TeLLAgent	SA	$1.83_{\pm 5}$	$20.71_{\pm 28}$	$31.64_{\pm 25}$	$3.35_{\pm 7}$	$179.65_{\pm 36}$	$3668_{\pm 2341}$	$262.99_{\pm 31}$
	ProSpect	$22.47_{\pm 13}$	$91.41_{\pm 12}$	$48.14_{\pm 30}$	$47.62_{\pm 30}$	$136.31_{\pm 54}$	$3005_{\pm 2994}$	$194.64_{\pm 59}$
Magentic-One	SA	$14.52_{\pm 10}$	$87.53_{\pm 6}$	$26.72_{\pm 15}$	$23.23_{\pm 14}$	$121.74_{\pm 32}$	$6030_{\pm 6007}$	$231.85_{\pm 48}$
	ProSpect	$24.91_{\pm 13}$	$91.65_{\pm 9}$	$42.79_{\pm 16}$	$39.23_{\pm 15}$	$101.58_{\pm 32}$	$12657_{\pm 11155}$	$204.03_{\pm 42}$
AILA	SA	$13.95_{\pm 13}$	$32.67_{\pm 25}$	$25.69_{\pm 22}$	$25.54_{\pm 22}$	$88.61_{\pm 45}$	$1094.15_{\pm 884}$	$98.37_{\pm 46}$
	ProSpect	$34.65_{\pm 14}$	$60.69_{\pm 12}$	$41.16_{\pm 14}$	$41.16_{\pm 14}$	$90.00_{\pm 36}$	$419.04_{\pm 504}$	$93.18_{\pm 37}$

Table 3: Magentic-One with the Actor fixed (DeepSeek V4 Pro 49B, thinking On) and the Speculator varied; all Speculators have thinking Off. Columns as in Table 2.

Speculator	Time	Next-agent	Content	Hit	Cost		
	saved (%)↑	match (%)↑	match (%)↑	rate (%)↑	Actor (+%)↓	Speculator↓	FLOPs (+%)↓
Qwen 9B	$18.58_{\pm 14}$	$75.43_{\pm 14}$	$44.19_{\pm 19}$	$41.11_{\pm 17}$	$62.99_{\pm 25}$	$13255_{\pm 12878}$	$236.14_{\pm 52}$
DeepSeek V4 Pro	$22.75_{\pm 14}$	$75.97_{\pm 15}$	$56.22_{\pm 17}$	$53.38_{\pm 16}$	$53.75_{\pm 24}$	$10217_{\pm 10621}$	$303.46_{\pm 66}$
Gemma 4 E2B	$17.25_{\pm 14}$	$77.36_{\pm 14}$	$38.54_{\pm 19}$	$35.96_{\pm 18}$	$68.30_{\pm 27}$	$11539_{\pm 11178}$	$215.28_{\pm 54}$

in-context learning, ProSpect also achieves superior content-match hit rate across all systems, which directly translates to superior combined hit rate.

Impact of Speculator. Two dimensions of the Speculator impact its performance: its parameter size and the LLM family it comes from. We investigate both.

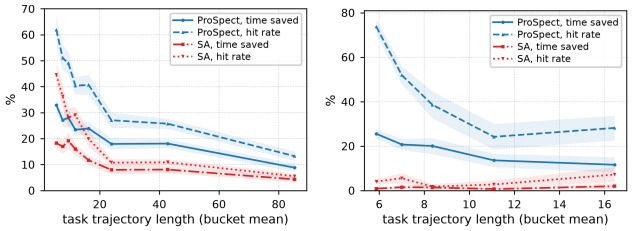
To measure the impact of parameter size, we hold the Actor fixed and move from a 9B to a 2B Speculator (Table 2(a) and (b)). We note that while next-agent match remains stable across all MASs, content match falls by a bigger margin of up to $\approx 16\%$. This is expected since transitioning to a smaller Speculator directly affects the message speculation quality. However, SA suffers a significant drop in next agent match since here the Speculator performs the next-agent prediction. Interestingly, the drop in latency gain due to moving to a 2B Speculator is not as significant because although the smaller Speculator does get a reduced content match, it is faster in speculating.

To measure impact of Speculator family, in Table 3 we hold the Actor fixed and swap the Speculator across three model families. We observe a strong signal that the Speculator should be chosen to match the Actor's family: the DeepSeek Speculator, drawn from the Actor's own family, gives both the highest hit rate (53.38%) and the lowest Actor and Speculator token cost.

Token Cost and Flops. Speculative acceleration buys latency with tokens and more compute. In this context, ProSpect always consumes less Flops than SA and costs less Actor tokens in 11 out of 12 cases since a higher hit rate means less discarded work. The speculator tokens, which is assumed to be much cheaper (or free) than Actor's, is sometimes higher than SA due to in-context learning.

4.3 Robustness

In Fig. 2, we measure the variation of time-saved and hit-rate against query complexity. We use the number of agentic communications required to solve the task (trajectory length) as a measure of complexity. Since speculation is harder for complex tasks, time saved is also lower. Nonetheless, ProSpect consistently achieves superior performance over SA.



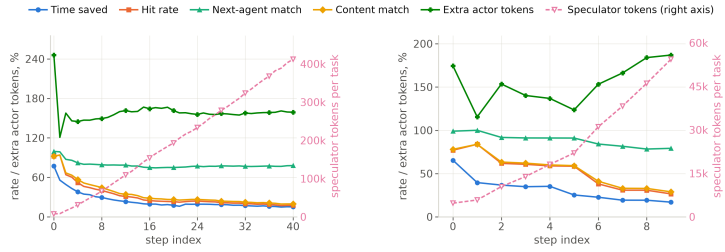
(a) Co-Sight (b) TeLLAgent

Figure 2: Time saved and hit rate against query complexity, measured in terms of trajectory length. Actor: Qwen 27B, Speculator: Qwen 9B. Results on other datasets are available in Fig. 6 in App. C.

In Fig. 3, we examine how the live performance evolves as the agentic trace progresses. Since errors can compound over successive steps, we generally expect performance to deteriorate as the trace grows. This trend is indeed observed, although the metrics largely saturate, with performance decreasing only gradually over additional steps.

4.4 Ablation Study and Parameters

We first isolate the contribution of the retrieval-augmented in-context learning (ICL) used by the message writer $\mathcal{S}_F$ (§3.3); Fig. 4 reports the results. ICL improves content match on every system, except Magnetic-One and the improvement is largest on TeLLAgent and AILA. This is the mechanism behind the domain effect noted in §4.2. Retrieval supplies past messages on the same transition, so it substitutes for a prior the Speculator does not have, and it therefore helps most where that prior is weakest. Magentic-One, whose components and tasks are the most generic of the four, is where the Speculator needs the least help.



(a) Co-Sight (b) TeLLAgent

Figure 3: Speculation quality along a trajectory, for Qwen 9B speculator. Results for other datasets are available in Fig. 7 in App. C.

How does the speculation cap W affect performance? Fig. 5 in Appendix C presents the results. All metrics increase monotonically with W, which means while we obtain improved latency, it comes at cost of more tokens. Hence, W acts as a knob to control the right trade-off as per one's needs.

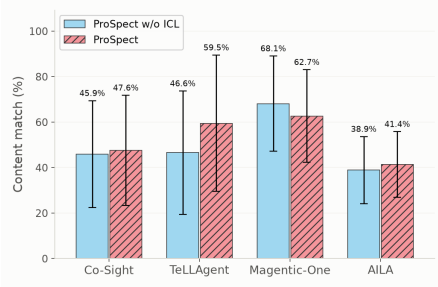


Figure 4: Effect of in-context exemplars on content match. The corresponding effects on hit rate and time saved are in Fig. 8.

5 Conclusions and Future Work

We presented ProSpect, which extends lossless speculative execution from single agents to multi-agent systems, where the execution trace is a DAG rather than a linear chain. ProSpect separates the two questions a speculation answers: which components are invoked next, cast as link prediction and answered by a recurrence folded along the execution DAG, and what each of them receives, left to the Speculator LLM; the budget is split between them adaptively, by the predictor's confidence. Across four systems and three LLM families, ProSpect saves 17.25 to 37.86% of wall-clock time and beats speculative actions (Ye et al., 2026), the state of the art for single-agent systems, on every system and LLM family. Notably, its advantage grows as the domain becomes less familiar to the Speculator, and is largest on the two scientific systems.

Future works and Limitations. Speculative acceleration trades tokens for latency, and we currently speculate at every round. Can the decision to speculate be made per round, as a function of predictor's confidence? Can the Actor's live reasoning trace, available before its call returns, sharpen our predictions mid-call? Our current study does not cover these questions, which we hope to study in the future.

AI USE STATEMENT

In this work, we used generative AI tools to obtain critical feedback on the design of our research questions and experimental protocol. Additionally, we used generative AI tools to draft and edit parts of this paper, to suggest its structure, and to create and revise figures.

We take responsibility for the final content of this work, including all text, claims, and artifacts produced with the aid of generative AI.

ETHICS STATEMENT

The acceleration reported in this work is bought with compute. Speculation issues up to W Actor calls at every step and commits at most one, so an accelerated run consumes more tokens, and therefore more energy, than the sequential run it replaces. The saving is in wall-clock time, not in resources. We regard this as the principal ethical consideration the paper raises, and we report token overhead alongside every latency number in §4 rather than reporting speedups alone, so that the trade is visible to anyone deciding whether to make it.

The work uses public benchmarks only and involves no human subjects or personal data.

REPRODUCIBILITY STATEMENT

Our codebase is available at `https://anonymous.4open.science/r/ProSpect`. The README includes scripts to reproduce all our experiments. All our datasets are drawn from publicly available repositories and benchmarks.

REFERENCES

Reyna Abhyankar, Qi Qi, and Yiying Zhang. OSWorld-Human: Benchmarking the efficiency of computer-use agents. In *Proceedings of Machine Learning and Systems*, volume 8, 2026.

Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *Transactions on Machine Learning Research*, 2024.

Dujian Ding, Ankur Mallick, Chi Wang, Robert Sim, Subhabrata Mukherjee, Victor Rühle, Laks V. S. Lakshmanan, and Ahmed Hassan Awadallah. Hybrid LLM: Cost-Efficient and Quality-Aware Query Routing. In *International Conference on Learning Representations*, 2024.

Alvaro Estebanez, Diego R. Llanos, and Arturo Gonzalez-Escribano. A survey on thread-level speculation techniques. *ACM Computing Surveys*, 49(2):22:1–22:39.

Adam Fourney, Gagan Bansal, Hussein Mozannar, Cheng Tan, Eduardo Salinas, Erkang Zhu, Friederike Niedtner, Grace Proebsting, Griffin Bassman, Jack Gerrits, Jacob Alber, et al. Magentic-one: A generalist multi-agent system for solving complex tasks. *arXiv preprint arXiv:2411.04468*, 2024.

Yichao Fu, Rui Ge, Zelei Shao, Zhijie Deng, and Hao Zhang. Scaling Speculative Decoding with Lookahead Reasoning. In *Advances in Neural Information Processing Systems*, 2025.

Yilin Guan, Qingfeng Lan, Fei Sun, Dujian Ding, Devang Acharya, Chi Wang, William Yang Wang, and Wenyue Hua. Dynamic Speculative Agent Planning. In *International Conference on Learning Representations*, 2026.

Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. In *International Conference on Learning Representations*, 2024.

Wenyue Hua, Mengting Wan, Jagannath Shashank Subramanya Sai Vadrevu, Ryan Nadel, Yongfeng Zhang, and Chi Wang. Interactive Speculative Planning: Enhance Agent Efficiency Through Co-Design of System and User Interface. In *International Conference on Learning Representations*, 2025.

Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. An LLM Compiler for Parallel Function Calling. In *International Conference on Machine Learning*, volume 235, pp. 24370–24391, 2024.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast Inference from Transformers via Speculative Decoding. In *International Conference on Machine Learning*, volume 202, pp. 19274–19286, 2023.

Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. Parrot: Efficient serving of LLM-based applications with semantic variable. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, 2024.

Indrajeet Mandal, Jitendra Soni, Mohd Zaki, Morten Smedskjaer, Katrin Wondraczek, Lothar Wondraczek, Nitya Gosvami, and N M Anoop Krishnan. Evaluating large language model agents for automation of atomic force microscopy. *Nature Communications*, 16, 10 2025. doi: 10.1038/s41467-025-64105-7.

Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: a benchmark for general AI assistants. In *International Conference on Learning Representations*, 2024.

Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M Waleed Kadous, and Ion Stoica. RouteLLM: Learning to Route LLMs from Preference Data. In *International Conference on Learning Representations*, 2025.

OpenAI. Introducing deep research. `https://openai.com/index/introducing-deep-research/`, February 2025. Accessed September 2026.

Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 15174–15186, 2024.

Samuel Schmidgall, Yusheng Su, Ze Wang, Ximeng Sun, Jialian Wu, Xiaodong Yu, Jiang Liu, Michael Moor, Zicheng Liu, and Emad Barsoum. Agent laboratory: Using LLM agents as research assistants. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pp. 5977–6043, 2025.

Jinyu Sun, Jibin Zhou, Huabei Wang, Wei Liu, Jun Yuan, Yue Wang, Ting Xie, Lin Tan, Hailiang Zhang, Yingping Zou, et al. TeLLAgent: a dual-agent framework for reliable scientific discovery with tool-enhanced LLMs. *Chemical Science*, 17(26):13101–13117, 2026.

Robert M. Tomasulo. An efficient algorithm for exploiting multiple arithmetic units. *IBM Journal of Research and Development*, 11(1):25–33, 1967.

Jikai Wang, Juntao Li, Jianye Hou, Yan Bowen, Lijun Wu, and Min Zhang. Efficient reasoning for LLMs through speculative chain-of-thought. In *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 1541–1553, 2026.

Naimeng Ye, Arnav Ahuja, Georgios Liargkovas, Yunan Lu, Kostis Kaffes, and Tianyi Peng. Speculative Actions: A Lossless Framework for Faster AI Agents. In *International Conference on Learning Representations*, 2026.

Guibin Zhang, Yanwei Yue, Zhixun Li, Sukwon Yun, Guancheng Wan, Kun Wang, Dawei Cheng, Jeffrey Xu Yu, and Tianlong Chen. Cut the Crap: An Economical Communication Pipeline for LLM-based Multi-Agent Systems. In *International Conference on Learning Representations*, 2025a.

Hongwei Zhang, Ji Lu, Shiqing Jiang, Chenxiang Zhu, Li Xie, Chen Zhong, Haoran Chen, Yurui Zhu, Yongsheng Du, Yanqin Gao, et al. Co-sight: Enhancing llm-based agents via conflict-aware meta-verification and trustworthy reasoning with structured facts. *arXiv preprint arXiv:2510.21557*, 2025b.

APPENDIX CONTENTS

A Proof of Proposition 2 **13**

B Implementation and Reproducibility Details **14**

B.1 Recording Protocol 14

B.2 Hardware Configuration 14

B.3 Software Configuration 14

B.4 Generation Settings and Codebase 15

B.5 Component Predictor Details 15

C Additional Experiments **15**

Table 4: Symbols used across more than one part of the paper. Quantities local to a single derivation, and the parameters of Table 5, are omitted.

Symbol	Meaning
System and trace (Defs. 1, 2)	
$\mathcal{M} = (\mathcal{A}, \mathcal{T}, C)$	MAS: agents, tools, coordination mechanism
$\mathcal{U} = \mathcal{A} \cup \mathcal{T}$	Components, $\mathcal{U} = \{1, \dots, n\}$; i, j denote components
q	Task
σ	Context fixed before execution
$G_q = (V_q, E_q)$	Execution trace on q; u, v are its nodes, each one invocation
i_u	Component invoked at node u
x_v	Input to node v
y_v, $y_{u \to v}$	Output of v, and the part of y_u routed to v
ℓ_v	Latency of node v
$\mathcal{P}(v)$	Parents of v
d	Depth of G_q
Latency (Eqs. 1, 4)	
r_v, $L(G_q)$	Ready time of v and end-to-end latency, without speculation
$\hat{c}_v$	Confirmation time: every parent of v has returned
$\hat{r}_v$, $\hat{e}_v$	Start and end time of v under speculation
$t_{u \to v}$	Time the correct message on (u, v) becomes available
$\hat{L}(G_q; \mathcal{S})$	End-to-end latency under Speculator $\mathcal{S}$
δ_u	Speculator latency at u
Speculation (Def. 4, Problem 1)	
$\mathcal{S}$	Speculator
$\widehat{E}_u$	Candidate set produced at u
m_u, k_u	Width on C and on the Actor; $m_u k_u \leq W$
W	Width cap
PROSPECT (§3)	
$\mathcal{S}_C$, $\mathcal{S}_F$	Component predictor and message writer
$\mathcal{H}$	Corpus of past traces; $\mathcal{H}(i, j)$ its edges on which i invoked j
$\mathcal{C}(u)$	Realized children of u
$\mathcal{N}(i)$	Components i invoked at least once in $\mathcal{H}$
$\mathcal{A}(u)$	Candidate set at u, $\mathcal{N}(i_u) \cup \{\perp\}$
$\perp$	Invoking nothing further
p_{uj}	Score for candidate j at u
$\mathbf{h}_j$	Learned embedding of component j
w_{ij}	Marginal inclusion rate of the transition $i \to j$
ϕ	Frozen text encoder
$\mathbf{g}_u$	Context at u
$\hat{i}$	A component predicted at u

A PROOF OF PROPOSITION 2

For $v \in V_q$ let d_v be the *depth* of v, the number of nodes on the longest path from a source of G_q to v, and let $d = \max_{v \in V_q} d_v$. Throughout, $\ell_v = \ell$ for all v, $\delta_u = 0$ for all u, and every edge of G_q is a hit.

Speculation never increases latency. We first show $\hat{e}_v \leq e_v$ for every v, with no assumption on the latencies. For a source, $\hat{e}_v = \ell_v = e_v$. For v with parents, $t_{u \to v} \leq \hat{e}_u$ in either branch of Eq. 3, so $\hat{r}_v \leq \max_u \hat{e}_u = \hat{c}_v$, and therefore $\hat{e}_v = \max(\hat{r}_v + \ell_v, \hat{c}_v) \leq \hat{c}_v + \ell_v$. By the inductive hypothesis $\hat{c}_v = \max_u \hat{e}_u \leq \max_u e_u = r_v$, hence $\hat{e}_v \leq r_v + \ell_v = e_v$. Taking maxima gives $\hat{L}(G_q; \mathcal{S}) \leq L(G_q)$, which is the right inequality of Eq. 6; it holds for any hit pattern and any $\delta_u \geq 0$. The remainder of this section establishes the left inequality, for which we take $\ell_v = \ell$ throughout.

Non-speculative latency. By Eq. 1, $e_v = \max_{u \in \mathcal{P}(v)} e_u + \ell$, so $e_v = d_v \, \ell$ by induction on d_v, and $L(G_q) = \max_v e_v = d \, \ell$.

Speculative latency. We show $\hat{e}_v = \lceil d_v/2 \rceil \, \ell$ by induction on d_v.

If $d_v = 1$ then v is a source, $\hat{c}_v = \hat{r}_v = 0$, and $\hat{e}_v = \ell = \lceil 1/2 \rceil \ell$.

Let $d_v > 1$ and write $D = \max_{u \in \mathcal{P}(v)} d_u = d_v - 1$. Since every edge is a hit and $\delta_u = 0$, Eq. 3 gives $t_{u \to v} = \min(\hat{e}_u, \hat{c}_u) = \hat{c}_u$, because $\hat{c}_u \le \hat{e}_u$ by Eq. 4. Hence, using the inductive hypothesis on the parents and their own parents,

$$\hat{c}_v = \max_{u \in \mathcal{P}(v)} \hat{e}_u = \left\lceil \frac{D}{2} \right\rceil \ell, \qquad \hat{r}_v = \max_{u \in \mathcal{P}(v)} \hat{c}_u = \left\lceil \frac{D-1}{2} \right\rceil \ell, \tag{13}$$

where both maxima are attained at a parent of depth D because $\lceil \cdot \rceil$ is nondecreasing, and $\hat{c}_u = 0$ for a source u agrees with $\lceil 0/2 \rceil \ell$. Therefore

$$\hat{e}_v = \max\left(\hat{r}_v + \ell,\ \hat{c}_v\right) = \ell \cdot \max\left(\left\lceil \frac{D-1}{2} \right\rceil + 1,\ \left\lceil \frac{D}{2} \right\rceil\right) = \left\lceil \frac{D+1}{2} \right\rceil \ell = \left\lceil \frac{d_v}{2} \right\rceil \ell. \tag{14}$$

The last step is a case check: for $D = 2k$ the two arguments are $k+1$ and k, and for $D = 2k+1$ they are $k+1$ and $k+1$; in both cases the maximum equals $\lceil (D+1)/2 \rceil$. Taking the maximum over v gives $\hat{L}(G_q; \mathcal{S}) = \lceil d/2 \rceil \ell$ in this best case. Since a larger δ_u or a miss can only delay $t_{u \to v}$ in Eq. 3, and every quantity in Eq. 4 is nondecreasing in $t_{u \to v}$, every other configuration is slower, so $\lceil d/2 \rceil \ell$ is a lower bound on $\hat{L}(G_q; \mathcal{S})$ for any Speculator, which is the left inequality of Eq. 6. Since $\lceil d/2 \rceil \ge d/2$, the reduction never exceeds $1/2$; it equals $1/2 - \frac{1}{2d}$ for odd d, so the bound is approached but not attained.

B Implementation and Reproducibility Details

B.1 Recording Protocol

For each task we run the Actor once through the system's native orchestration, unchanged, and record the ordered component invocations, their messages and tool calls, token counts, and per-step wall-clock latency. Every method is then replayed against this recording, so the Actor's decisions and timings are identical across methods for fair comparison. The Speculator is executed live during replay, so the Speculator latency δ_u of Def. 4 is measured rather than assumed.

Stopping rules. GAIA and TeLLAgent runs end when the native system returns its final answer. For TeLLAgent we use its controlled tool-calling suite, which varies the number of required sequential calls and the size of the available tool set, rather than end-to-end discovery runs. AILA recording ends after the first successful consequential action, or when the system finishes without one, so AILA latencies are measured over a prefix of the trace rather than a completed task.

Sandboxing. AILA instrument commands are checked against a validated Nanosurf mock and never reach physical hardware. Magentic-One follows the AutoGenBench setup, in which file and code operations run in a fresh Docker container per task.

B.2 Hardware Configuration

All experiments were conducted on the same host. The configuration and the resources assigned to each run were:

- **CPU:** 96 logical cores, of which eight were allocated to each run;
- **RAM:** 512 GB; and
- **GPU:** one NVIDIA A100-PCIe-40GB per run.

This allocation was held fixed across all systems and configurations.

B.3 Software Configuration

- **Operating system:** Ubuntu 20.04.4 LTS (GNU/Linux 5.4.0-124-generic, x86_64);
- **Model serving:** Ollama 0.32.6;

- **PyTorch:** 1.13.1+cu117 with CUDA 11.7; and
- **PyTorch Geometric:** 2.3.1.

Ollama served the Actor and the Speculator. PyTorch and PyTorch Geometric were used for the component predictor $\mathcal{S}_C$.

B.4 Generation Settings and Codebase

The Actor used a maximum generation length of 2,048 tokens, temperature zero, and reasoning enabled. The Speculator used a maximum of 2,048 tokens, temperature 0.7, and reasoning disabled. Both used a 65,536-token context window. These settings and the coverage target τ of §3.2 were held fixed over every task in a test set.

The codebase separates system integration from speculation and evaluation. A system-specific adapter records native agent and tool events without changing the system's orchestration policy. A shared trace layer stores the ordered component calls, messages, tool calls, token counts, and measured latencies. A common replay module runs the Speculator against these traces, and a common evaluator reconstructs time savings and token overhead. All four systems use this pipeline; only the adapter that maps native component names and events into the common trace format is system specific.

B.5 Component Predictor Details

Optimization. We train with AdamW for 300 epochs at learning rate 10^{-4}, weight decay 10^{-3}, and batch size 16 under a cosine-annealing schedule, minimizing Eq. 8, and select the checkpoint with the lowest validation loss.

Default parameter values. Table 5 lists the values used throughout unless a study varies them.

Table 5: Default parameter values.

Symbol	Meaning	Value
W	Speculation width cap	3
τ	Coverage target on the normalized scores	0.8
κ	top-κ retrieval budget	1
d_h	Component embedding dimension	64
–	Recurrence layers / width / dropout	2 / 256 / 0.3
–	Scoring network hidden width	256
–	Task embedding dimension after projection	128

Evaluation protocol and leakage control. We evaluate under 10-fold cross-validation at a fixed seed. In each fold, eight parts are used for training, one for validation, and one for testing, and we report the mean and standard deviation over the ten folds. The retrieval corpus of §3.3 is assembled from recorded Actor traces, one entry per invocation, and is restricted to the components that carry out the task. It excludes every invocation drawn from a task in the same fold as q, under the same partition used to train and evaluate the component predictor, so a demonstration never originates in a task the predictor was tested on. When the pool of §3.3 is empty, the context falls back to a fixed default example written once per component, which is also the example used by the static in-context baseline.

C Additional Experiments

This appendix reports the full results behind the analyses summarised in §4.3 and §4.4. All results use the `qwen3.5:9b` Speculator unless stated otherwise.

Speculation budget W. Fig. 5 shows that time saved, hit rate, and content match rise with W across systems. Next-agent match stays nearly flat on Co-Sight and TeLLAgent, so the extra candidates go mainly to alternative message contents. Both extra Actor tokens and Speculator tokens, by contrast,

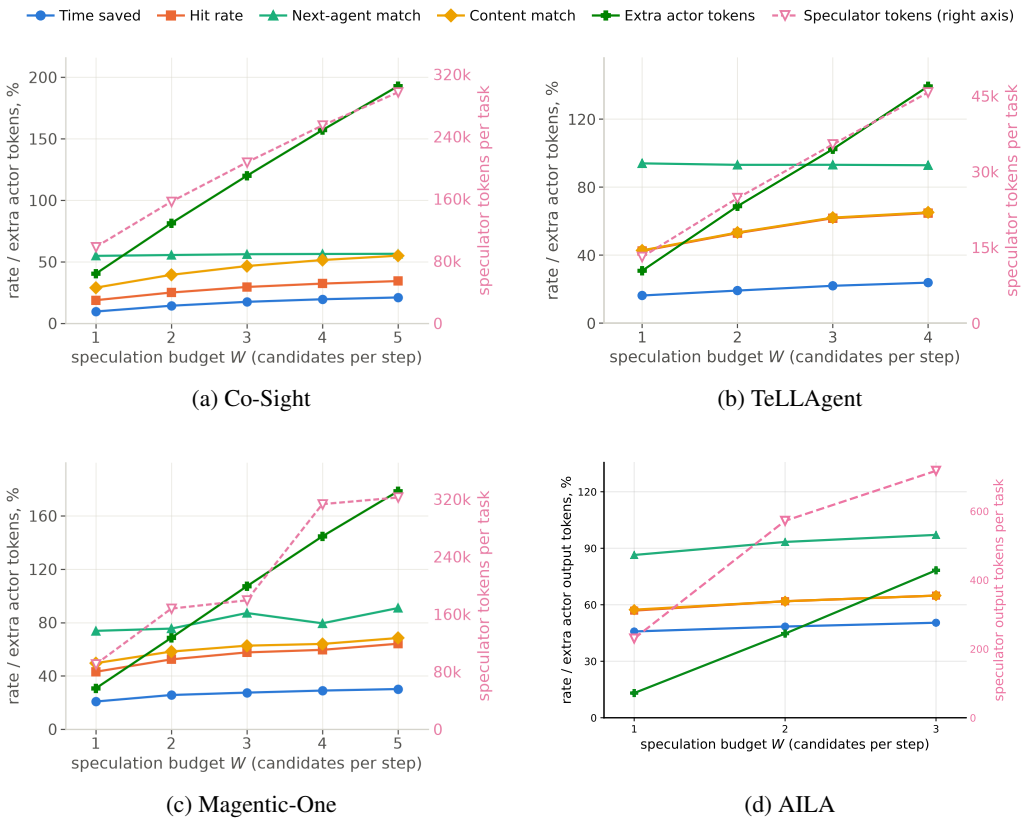


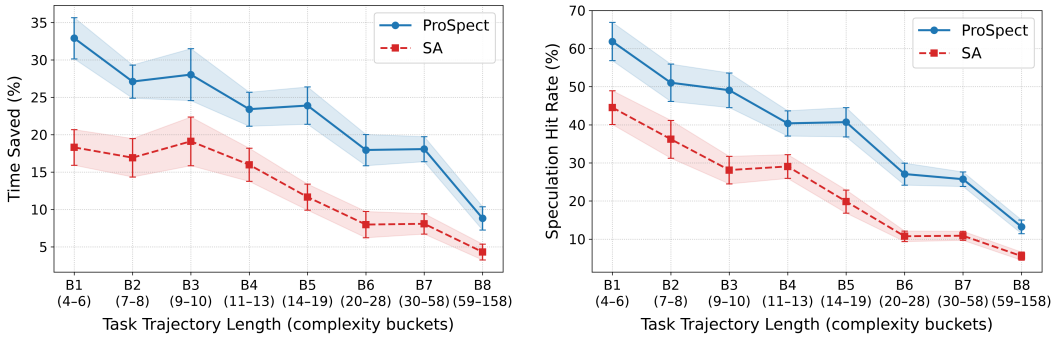
Figure 5: Impact of speculation budget W on the various metrics of importance on `qwen3.5:9b` speculator. Note that, Speculator output tokens here are total over all steps.

grow roughly linearly in W. Small values of W therefore capture most of the achievable savings at a fraction of the cost.

Query complexity. Fig. 6 extends Fig. 2 to all four systems. Both metrics decline as trajectories grow longer, yet ProSpect outperforms SA in every bucket. The gap is most pronounced on TeLLAgent, where SA saves at most $\approx 2\%$ of wall-clock time while ProSpect saves 12–26%. On Magentic-One the two methods are close on short tasks, but the gap widens with complexity ($\approx 22\%$ vs. $\approx 12\%$ time saved in the longest bucket).

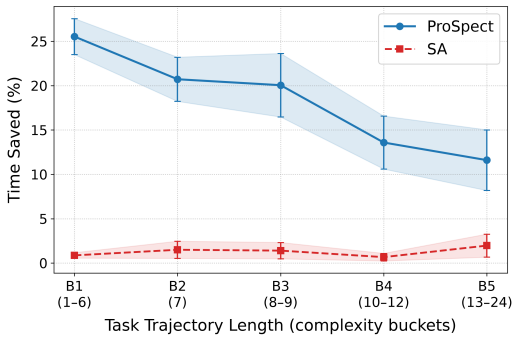
Performance along a trajectory. Fig. 7 extends Fig. 3. On Magentic-One, next-agent match stabilises at $\approx 92\%$ within a few steps. The decline in hit rate and time saved is therefore driven by content prediction, and it occurs mostly within the first $\sim$10 steps before flattening. Speculator tokens grow linearly with the number of steps because we speculate at every round, which motivates the per-round speculation decision discussed in the future work.

Effect of ICL. Fig. 8 complements Fig. 4. ICL improves hit rate on all systems except Magentic-One. The gain is largest on TeLLAgent ($46.1\% \rightarrow 56.6\%$) and AILA ($38.6\% \rightarrow 41.4\%$). This is the mechanism behind the domain effect noted in §4.4 and §4.2. Time saved never decreases with ICL. It improves most on TeLLAgent ($+2.8\%$) and AILA ($+2.3\%$), and is essentially unchanged on Co-Sight and Magentic-One. Hit-rate gains translate only partially into time saved, since a hit reduces latency only when the speculated call lies on the critical path.

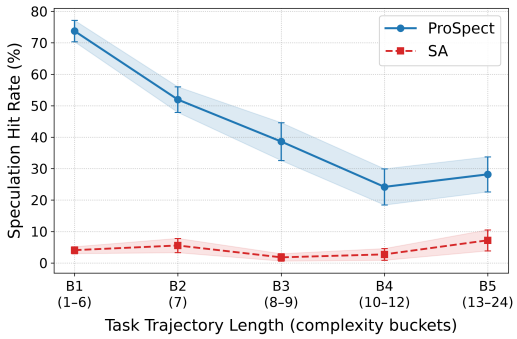


(e) Co-Sight: time saved

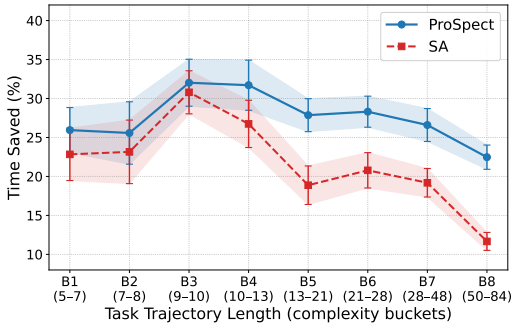
(f) Co-Sight: hit rate



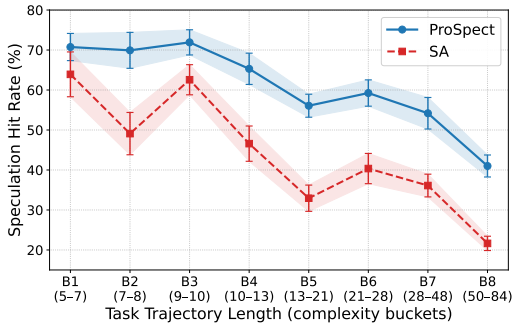
(g) TeLLAgent: time saved



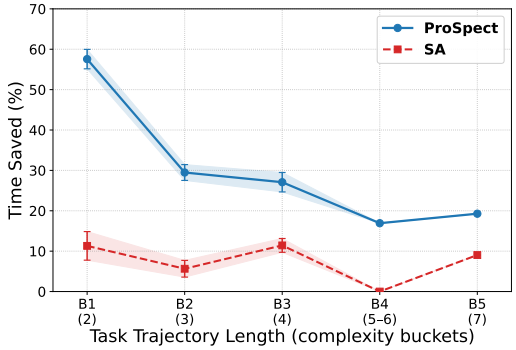
(h) TeLLAgent: hit rate



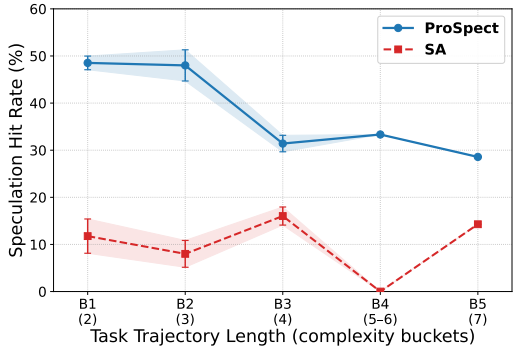
(i) Magentic-One: time saved



(j) Magentic-One: hit rate

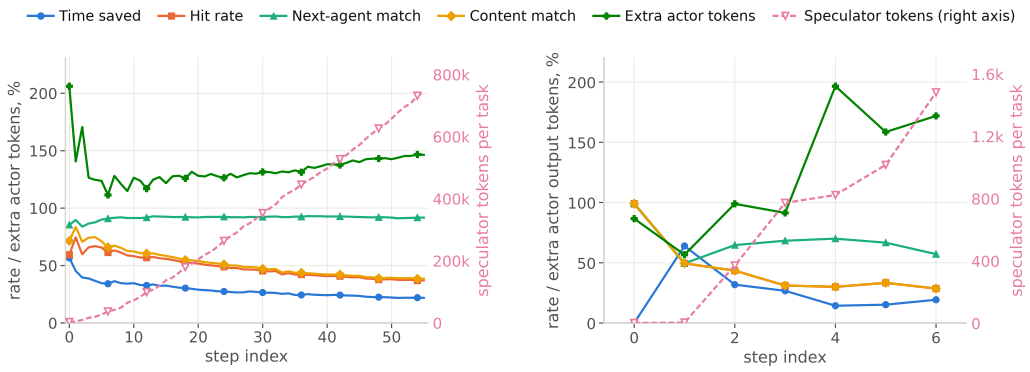


(k) AILA: time saved



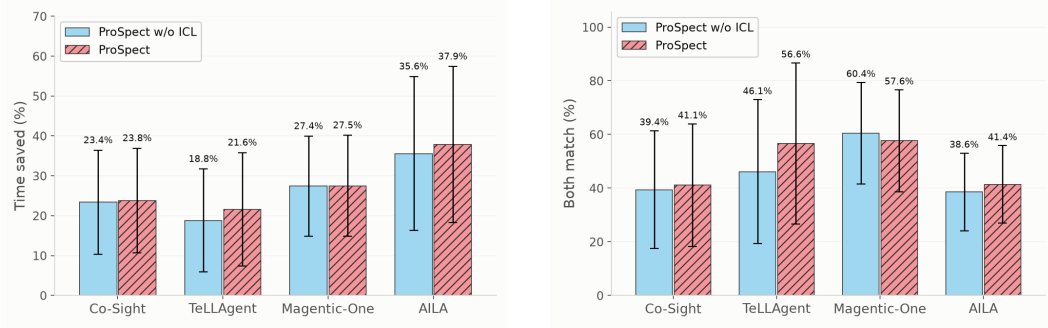
(l) AILA: hit rate

Figure 6: Time saved and speculation hit rate (next agent and content both correct) against query complexity, measured in terms of trajectory length.



(a) Magentic-One (b) AILA

Figure 7: Cumulative performance and cost over a task's steps for PROSPECT with the Qwen 9B speculator. Rates and extra actor tokens (left axis, %) are cumulative over steps $0..t$; speculator tokens (right axis) are the running total billed per task.



(a) Time saved (b) Hit rate

Figure 8: Effect of in-context exemplars on time saved and hit rate, companion to Fig. 4. Each pair is the best arm of its group by live time saved, with and without ICL; error bars are the s.d. across questions.